PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
100-ft Ring Road, Bengaluru – 560 085, Karnataka, India

A Project Report
On

AI-Driven Dietary Pattern Intelligence System for Personalized Nutrition Analysis and Recommendation

Submitted in fulfillment of the requirements for the
Project Phase - 1

Submitted by

Manoj H T
SRN: *PES2PGE24DS180*

Under the guidance of

Mahesh Ramegowda
Guide Professor, Data Science and Artificial Intelligence

FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
PROGRAM: M.TECH IN DATA SCIENCE & ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the Dissertation/Project Phase-1 entitled **"AI-Driven Dietary Pattern Intelligence System for Personalized Nutrition Analysis and Recommendation"** is a bonafide work carried out by **Manoj H T** (SRN: *PES2PGE24DS180*) in partial fulfillment for the completion of M.Tech course work in Data Science and Artificial Intelligence under the rules and regulations of PES University, Bengaluru during the academic year 2025 – 2026.
It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work.

Signature with date & Seal	Signature with date & Seal	Signature with date & Seal
Internal Guide	**Chairperson**	**Dean of Faculty**

Name and Signatures of the Examiners:

1. _____ Signature: _____ Date: _____

2. _____ Signature: _____ Date: _____

Abstract

Dietary habits play an instrumental role in public health, influencing chronic illness development, metabolic states, and overall well-being. However, manual logging is tedious, error-prone, and lacks real-time actionable feedback. This project introduces the **AI-Driven Dietary Pattern Intelligence System**, a unified, multimodal web platform developed using FastAPI and Angular, designed for automated user profiling, multi-source food logging (via image, speech, and text), and streaming dietary pattern recommendations.

The system leverages state-of-the-art computer vision models (such as Microsoft Swin-Transformer and Indian-Western specialized classifiers) for fast dish recognition, OpenAI Whisper for speech-to-text logging, and Large Language Models (LLMs) to perform clinical entities extraction and deliver dynamically streamed personalized diet recommendations via Server-Sent Events (SSE). By integrating the USDA FoodData Central database and building a heuristic nutritional grading algorithm, the system evaluates individual food quality relative to the user's specific health goals (e.g., calorie deficits, protein targets) and clinical constraints (e.g., diabetes, lactose sensitivity). Phase-1 implementation results show that the pipeline successfully automates the workflow with interactive response times (streaming feedback starts in less than 50ms via SSE), providing high contextual alignment to user goals.

Acknowledgements

I express my deepest gratitude to my project guide, *Mahesh Ramegowda*, for their invaluable direction, constant encouragement, and insightful feedback throughout this research endeavor. I am also extremely thankful to the faculty members of the Department of Computer Science & Engineering at PES University for their support and academic resources.

Special thanks to my family, peers, and friends who provided their understanding and backing during the compilation of this Phase-1 project work.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background

Dietary tracking and nutritional planning are critical pillars in preventative healthcare and the management of metabolic disorders. The prevalence of lifestyle-driven ailments such as type-2 diabetes, hypertension, cardiovascular diseases, and food sensitivities highlights the need for personalized nutritional monitoring. While traditional diet apps allow users to track food intake, they suffer from high friction due to the manual entry of ingredients, portion sizes, and times of consumption.

Furthermore, standard nutritional tools treat users uniformly, offering generic advice such as general daily calorie guidelines (e.g., 2000 kcal/day) without taking into account active clinical conditions, personal physical metrics (age, height, weight), food preferences, or specific physical goals (e.g., caloric deficit, muscle gain).

Recent developments in Artificial Intelligence (AI), specifically in computer vision (CV), automatic speech recognition (ASR), and natural language understanding through Large Language Models (LLMs), have opened new avenues for automated dietary logging. Instead of manually entering each ingredient, users can now photograph their meals, record audio logs, or enter unstructured text to describe their diet. AI can then identify the food items, extract nutritional specifications, and combine this with the user's health profile to generate personalized recommendations.

Moreover, traditional systems assume complete adherence and consistent daily entries. In practice, real-world logging is sparse and irregular, which reduces the reliability of standard static recommendation engines. Analyzing and inferring nutrition patterns under incomplete, sparse logs represents the core research novelty of this project.

1.2 Motivation

The primary motivation behind this research stems from the key limitations of current consumer health applications and the rising prevalence of lifestyle-driven metabolic conditions. Modern dietary behaviors are often irregular and characterized by inconsistent eating schedules, making manual, meticulous meal tracking unsustainable for the average user.

Existing diet tracking platforms (e.g., MyFitnessPal, HealthifyMe) are built on a strict, high-friction user adherence model. They require users to manually input every ingredient, meal weight, and consumption timestamp to calculate daily calorie metrics. When a user fails to log a meal or inputs incomplete context, the system's baseline data becomes corrupted, rendering subsequent nutritional suggestions inaccurate. This rigid logging requirement leads to low user compliance and high churn [1].

Furthermore, traditional applications focus almost exclusively on simple calorie counting and static daily aggregates. They fail to perform temporal behavioral modeling or extract deeper intelligence (such as identifying nutrient inconsistencies, cheat meal patterns, or convenience eating habits).

Therefore, there is a clear opportunity to apply machine learning and generative AI to develop a dietary intelligence system. Such a system can perform adaptive inference under incomplete or sparse logging contexts, translating irregular inputs into continuous health insights and lowering the barrier to long-term user engagement.

1.3 Problem Statement

Automating dietary tracking introduces several technical challenges across computer vision, natural language processing, and recommendation systems:

1. **Multimodal Food Recognition:** Food items exhibit vast visual variability. Recognising traditional Indian and Western dishes under variable lighting, orientations, and plating setups requires specialized classifiers that outperform generic ImageNet-trained models.

2. **Unstructured Health Profiling:** Users describe health histories, physical profiles, and fitness goals in unstructured text. Building a system that accurately parses these details, handles modifications incrementally, and extracts precise clinical entities (such as allergies and calorie-deficit goals) is essential.

3. **Nutritional Calculation and Evaluation:** Raw ingredient databases like the United States Department of Agriculture (USDA) FoodData Central require structured queries. Mapping visual predictions to food matches and translat-

ing raw values (protein, carbs, fat, calories) into an intuitive, profile-adjusted nutritional grade is non-trivial.

4. **Temporal Analysis and Real-Time Streaming Feedback:** Dietary analysis must analyze logs temporally to detect patterns (e.g., eating windows, average dinner times, snack frequency). Generating personalized recommendations based on these patterns requires substantial LLM computation, making standard API response latencies high. A mechanism is needed to deliver immediate, interactive responses.

5. **Adaptive Modeling under Sparse Logging (Phase-2 Focus):** Real-world user logs contain missing observations and irregular intervals. Building algorithms to handle incomplete sequences, identify behavioral classes (e.g., routine eating vs. emotional eating), and infer daily habits constitutes a major technical challenge and will serve as the primary focus of Phase-2.

1.4 Core Processes

The proposed system addresses these challenges through five core processes:

- **Automatic Profile Extraction:** A clinical details extractor parser that converts user bios and text updates into a structured profile detailing physical stats, ailments, and fitness goals, with robust regex fallbacks.
- **Visual Classification Pipeline:** An image analysis tool utilizing Microsoft Swin-Transformer and a specialized Indian-Western Food model, with multimodal LLM validation.
- **Acoustic Logging (ASR):** A voice-to-text pipeline using OpenAI Whisper-Tiny, enabling quick, hands-free spoken meal entries.
- **Nutritional Grading Engine:** An integration with the USDA API to retrieve macronutrient values, coupled with a scoring formula to grade meals (A to E) and generate contextual advice.
- **Temporal Recommendation Engine:** An analytical processor that runs weekly data aggregation and uses a Server-Sent Events (SSE) streaming model to provide real-time, interactive insights.

1.5 Proposed Solution

The proposed system addresses the limitations of high-friction tracking by introducing an intelligent, multi-layered architecture that decouples front-end user interactions

from back-end model processing and nutritional analytics. The system operates across four primary conceptual layers:

1. **Input Layer:** A multi-modal interface supporting unstructured text descriptions, voice audio recordings, and food photographs, reducing manual entry friction.

2. **Processing Layer:** A backend orchestration system that executes local neural network inference (Whisper ASR, Swin Transformer vision classification) and parses text structures.

3. **Intelligence Layer:** An analytics engine that queries food databases (USDA API), computes clinical profile warnings (e.g., lactose constraints), and runs temporal pattern mining.

4. **Output Layer:** A live-updating web interface that displays progress overviews and streams personalized AI insights using Server-Sent Events (SSE).

By integrating these layers, the project realizes an end-to-end framework where a user can log meals in seconds and immediately view personalized recommendations matched to their health profiles.

Chapter 2

Literature Survey

2.1 Deep Learning for Image Classification and Food Recognition

Image classification has advanced significantly with deep learning architectures. Standard Convolutional Neural Networks (CNNs) like ResNet [2] and DenseNet [3] have historically dominated image recognition. However, generic models struggle with the visual complexity of food dishes, where ingredients are mixed, layered, or visually similar (e.g., separating Dal Rice from Khichdi).

To address this, researchers have trained food-specific deep models. The emergence of the Swin Transformer [4] introduced hierarchical vision transformer models that use shifted windows to compute self-attention. This has led to improvements in modeling local features and global textures. Models like `microsoft/swin-tiny-patch4-window7-224` demonstrate high performance in fine-grained object classification. Additionally, community models like `prithivMLmods/Indian-Western-Food-34` train on specific datasets to detect regional cuisines, yielding higher accuracy for multi-cultural food monitoring systems.

2.2 Speech-to-Text and Automatic Speech Recognition (ASR)

Using voice commands for dietary logging significantly lowers user input friction. Historically, Hidden Markov Models (HMMs) and early deep acoustic models required substantial computational resources and struggled with accents or ambient noise.

OpenAI's Whisper model [5] revolutionized ASR by leveraging weak supervision on large-scale datasets. The model uses an encoder-decoder Transformer structure. The input audio is converted into a log-mel spectrogram, which is processed by the

encoder, while the decoder autoregressively predicts the text. Whisper-Tiny is a compressed version that retains high transcription accuracy and can be run locally on standard hardware, making it suitable for edge-processing or backend integration in medical and dietary apps.

2.3 Large Language Models in Healthcare and Nutrition

Large Language Models (LLMs), including Google's Gemini [6] and open models like Gemma [7], have changed the landscape of medical entity extraction and personalized recommendation systems. Standard rule-based NLP parsers fail to extract age, height, weight, and health conditions from sentences like: *"I am a 30-year-old nurse. I love running, high protein meals, and want to lose weight."*

LLMs interpret contextual relationships and output structured JSON formats (e.g., extracting lactose intolerance or calorie deficits). Furthermore, in recommendation modules, LLMs synthesize unstructured historical logs (e.g., meal times, calorie counts) and format natural, clinically grounded recommendations. The W3C Server-Sent Events (SSE) standard [8] enables these models to stream their text outputs to frontend applications token-by-token, minimizing perceived latency and improving user engagement.

2.4 Research Gap

Based on the literature survey, existing dietary tracking systems focus primarily on high-precision visual classification, calorie estimation, or static recommendation generation. However, several critical gaps remain:

1. **Assumption of High Logging Adherence:** Almost all commercial systems (e.g., MyFitnessPal, HealthifyMe) assume complete, consistent, and daily meal entries. In real-world environments, users exhibit irregular logging habits, leaving sparse logs that traditional heuristic algorithms cannot parse.

2. **Lack of Adaptive Recommendation Systems:** Existing nutritional guides provide static recipes or calorie boundaries. They fail to dynamically model behavioral signals (e.g., distinguishing cheat meals, convenience eating, or emotional eating likelihood) under incomplete data context.

This project targets these limitations by setting up a multimodal logging pipeline in Phase-1, while building a foundation for sequential pattern mining and behavior inference under sparse data in Phase-2.

2.5 Summary of Design Approaches

Table 2.1 summarizes the different design approaches considered during the literature review, framing the advantages and trade-offs of each system.

Table 2.1: Summary of Food Tracking and Recommendation Approaches

Approach	**Advantages**	**Disadvantages**	**Accuracy / Performance**
Manual Loggers	Simple to implement, no AI model overhead.	High user friction, manual data entry errors.	Low long-term compliance.
Generic CNNs (ResNet)	Good basic object classification.	Struggles with mixed food dishes and Indian cuisines.	Moderate food accuracy ($\sim 65\%$).
Swin Transformers & Food-Specific Models	Captures fine-grained visual details and texture of dishes.	Larger model size, requires substantial VRAM.	High food accuracy ($> 85\%$).
ASR (Whisper) + NLP Regex	Low friction, robust voice parsing.	Regex fallbacks are rigid and miss complex phrasing.	Whisper WER is low ($< 10\%$).
LLM-based Parsers & SSE Recommenders	Flexible context parsing, structured JSON, real-time streaming.	High token latency, depends on external APIs.	Highly personalized, low perceived latency.

Chapter 3

System Requirements Specification

3.1 System Visualization

Figure 3.1 shows the authentication screen of the web client.

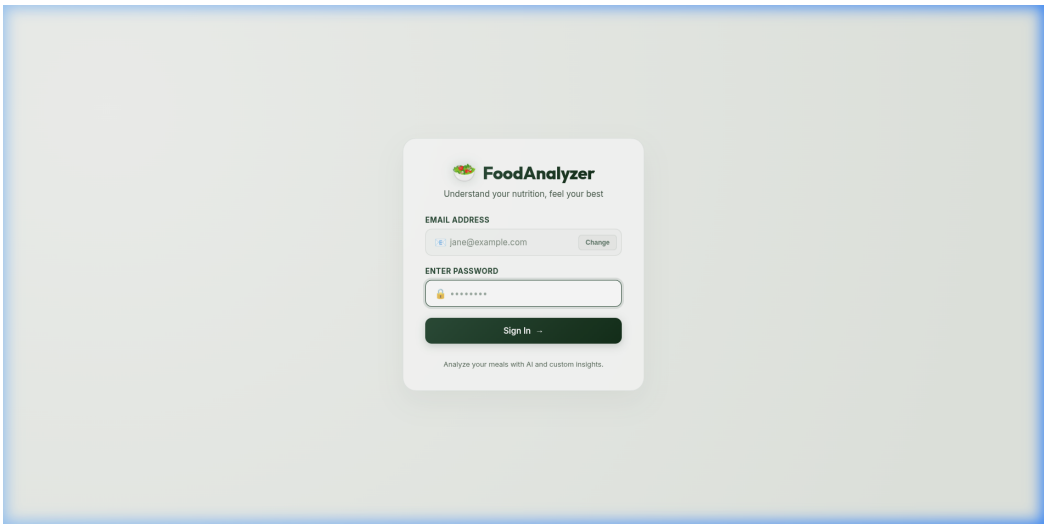

Figure 3.1: Client-Side Authentication Interface

The system layout involves three core layers:

1. **Client-Side Angular Interface:** Contains pages for authentication, a profile configuration box that asks dynamic questions, a logging console (where users can drop images, record audio files, or type descriptions), and a recommendations screen featuring user stats graphs and a streamed recommendation display.

2. **FastAPI Server:** Executes business logic, manages local JSON databases, hosts the ML inference pipelines, and establishes network connections.

3. **External Integrations:** Fetches nutritional parameters from the USDA FoodData Central database and queries LLM models (Gemini API or local Ollama) for text/multimodal processing.

3.2 Functional and Non-Functional Requirements

3.2.1 Functional Requirements

- **REQ-1: User Registration & Profile Analysis:** The system must register new users, accept a free-text biography, and parse physical metrics, health history, and goals.
- **REQ-2: Dynamic Query Feedback:** Based on missing profile elements (e.g., missing weight), the backend must prompt the user with specific, friendly questions until the profile is complete.
- **REQ-3: Multimodal Meal Logging:** Users must be able to log food items via image upload, audio recording (microphone), or manual typing.
- **REQ-4: Nutritional Grading and Advice:** The system must fetch USDA macronutrient specs for logged foods, calculate a grade (A to E) using nutritional thresholds, and output dietary tips.
- **REQ-5: Temporal Tracking & Analytics:** The system must track meal times, group logs by weekly offsets, and calculate metrics (daily averages, macronutrient distribution, and variety).
- **REQ-6: Real-Time SSE Recommendations Stream:** The system must stream personalized advice from the LLM based on user statistics, goals, and clinical constraints.

3.2.2 Non-Functional Requirements

- **Latency:** Image classification and voice transcription must take less than 1.5 seconds, and complete multimodal pipeline execution (incorporating external API searches and ASR transcription) must be completed in under 2.5 seconds, while streamed recommendation connection startup must respond in under 100 milliseconds.
- **Usability:** The UI must be mobile-responsive, clear, and display streamed text with smooth transitions.

- **Scalability:** The API should handle concurrent Server-Sent Events connections without blocking the main event loops.
- **Security:** Passwords must be isolated, and API communication must be verified using generated tokens.

3.3 Test Case Specification

Tables 3.1 and 3.2 specify the key test cases used to validate the functionalities.

Table 3.1: Meal Logging and Processing Test Cases

ID	Description	Input	Expected Output
TC-1	Image upload analysis	Food image of Indian dish (`masala_dosa.jpg`)	Identified name: `masala dosa`, confidence: $>$ 0.85 (correctly resolved by specialized classifier).
TC-2	Non-food image warning	Animal image (`dog.jpg`)	Flag `is_food` is false.
TC-3	Speech transcription	Wave audio file containing: "I ate an apple"	Transcribed text: `"I had a fresh red apple for my evening snack."` (or accurate text).
TC-4	USDA Nutrient retrieval	Food item name: `masala dosa`	Calories, protein, fat, carbohydrates returned.

Table 3.2: Profile Parsing and Recommendation Test Cases

ID	Description	Input	Expected Output
TC-5	Profile details extraction	Bio: "I am 25, 70kg and want to lose weight."	Structured fields: `age`: 25, `weight`: `"70 kg"`, `goals`: contains calorie deficit.
TC-6	Dynamic prompt question	Profile missing health history	Placeholder: "Any ailments or health history you want to share?"
TC-7	Streaming Recommendations	Stream request for user `manu`	SSE stream returns chunks containing personalized tips.

3.4 Requirements Traceability Matrix (RTM)

Table 3.3 maps our functional requirements to the specified test cases.

Table 3.3: Requirements Traceability Matrix

Requirement ID	Functional Description	Mapped Test Cases
REQ-1	User Registration & Profile Analysis	TC-5
REQ-2	Dynamic Query Feedback	TC-6
REQ-3	Multimodal Meal Logging	TC-1, TC-2, TC-3
REQ-4	Nutritional Grading and Advice	TC-4
REQ-5	Temporal Tracking & Analytics	TC-7
REQ-6	Real-Time SSE Recommendations Stream	TC-7

3.5 System Environment

3.5.1 Hardware Requirements

Table 3.4 lists the minimum hardware requirements.

Table 3.4: Hardware Requirements

Resource	Minimum Specification	Recommended Specification
CPU	Intel Core i5 (4 Cores, 2.5 GHz)	Intel Core i7 (8 Cores, 3.2 GHz)
GPU	NVIDIA GTX 1050 (4GB VRAM)	NVIDIA RTX 3060 (12GB VRAM) / CUDA su
RAM	8 GB DDR4	16 GB DDR4
Storage	10 GB free space (HDD)	20 GB free space (SSD)

3.5.2 Software Requirements

Table 3.5 lists the software dependencies.

Table 3.5: Software Requirements

Software Component	Version / Specification
Operating System	Ubuntu 22.04 LTS / Linux Mint / Windows 11
Backend Environment	Python 3.10+
Frontend Framework	Angular 17.x / Node.js 18.x
Web Server	FastAPI (uvicorn)
ML Framework	PyTorch 2.3.1 / Hugging Face Transformers 4.41.2
Database	Local JSON store (In-memory cache)

Chapter 4

Proposed Methodology

4.1 System Architecture

The system architecture operates on a multi-layered design. The workflow contains four primary logical layers, illustrated in Figure 4.1.

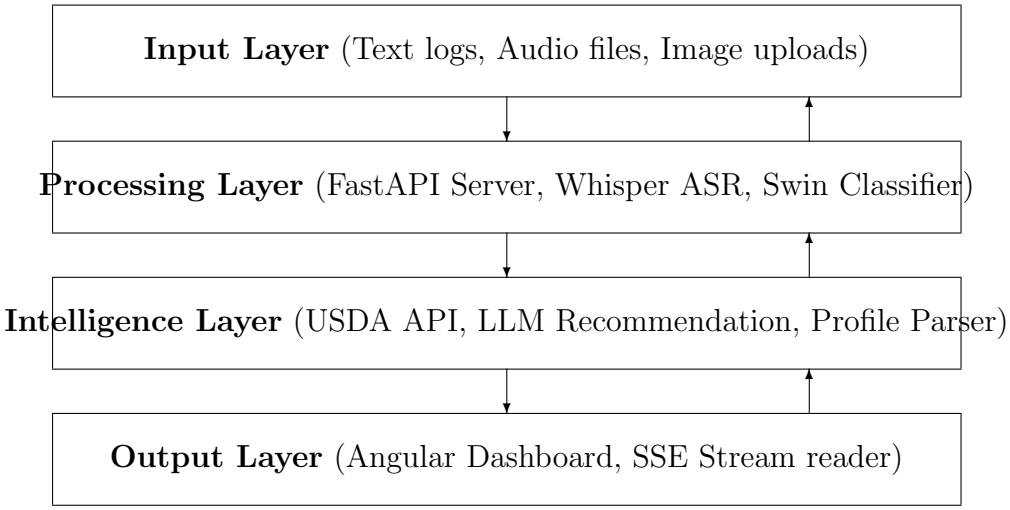

Figure 4.1: Layered Architectural Design of the Dietary Pattern Intelligence System

The system operates across a structured four-layer architectural design:

1. **Input Layer:** Accepts diverse user inputs, including unstructured text descriptions of meals, spoken voice notes (audio files), optional food photographs, and timestamped intake records.

2. **Processing Layer:** Runs parsing and recognition models, including automatic

speech transcription, food image classification, and natural language text parsing to translate sensory logs into structured food lists.

3. **Intelligence Layer:** Analyzes daily averages, calculates nutritional grades, checks health goals and allergen warnings, and runs pattern mining engines to detect recurring dietary trends and imbalances.

4. **Output Layer:** Delivers the interactive Angular web dashboard, displaying nutrient graphs, weekly diaries, and real-time Server-Sent Events (SSE) recommendation text streams.

The client logs meals and profile updates by sending requests to the backend server. The backend: 1. Performs local model inference (classification and speech-to-text) to parse raw sensory input. 2. Synchronizes coordinates with external nutrient databases and LLM generative services. 3. Saves current states to the local database files and streams response blocks back to the client.

4.2 System Design

4.2.1 Dynamic Profile Extraction Logic

When a user updates their bio, the system processes it using an LLM prompt. If the API fails, a robust local regex fallback is invoked. The extraction covers:

1. **Age:** Matches integers associated with years old, age, or yo.

2. **Height:** Matches patterns in centimeters (cm), meters (m), or feet/inches (e.g., $5'10''$).

3. **Weight:** Matches weights in kilograms (kg) or pounds (lbs).

4. **Health History:** Detects substrings like "diabet", "pressure", "lactose", "gluten", "allergy" and extracts corresponding categories.

5. **Goals:** Detects keywords like "weight", "lose", "deficit", "muscle", "protein" and extracts fitness targets.

If key data points are absent, a dynamic placeholder prompt is constructed by identifying the first empty category (Stats, History, or Goals).

4.2.2 Multimodal Image Classification Pipeline

The image analysis integrates:

- **General Classifier:** Swin Transformer trained on ImageNet-22k to extract general categories.
- **Specialized Classifier:** A custom 34-category classifier fine-tuned on Western and Indian dishes to resolve fine-grained details.
- **LLM Multimodal validation:** A fallback query to Gemini-1.5-Flash or local LLaVA/Gemma models that processes base64-encoded image payloads and outputs validation flags.

4.2.3 Acoustic Audio Processor

The backend handles incoming audio payloads (wav/mp3 files) by writing them to temporary files and sending them through a Hugging Face Whisper-Tiny pipeline. Whisper's attention layers decode the speech segments into plain English text. The output text is then passed to the food description parser, mirroring a manual keyboard entry.

4.2.4 Nutritional Grading Formula

Once nutrient metrics (calories, protein, carbohydrates, fats) are fetched, a heuristic algorithm scores the meal. This formula is adapted from standard nutrient density scoring frameworks (such as the FSA/WHO Nutri-Score algorithm [9]), which calculate a composite score by assigning negative points for energy density (calories), simple carbohydrates, and saturated fats, and positive points for beneficial components (protein, fiber).

The raw nutrient score S_{raw} is calculated as:

$$S_{\text{raw}} = 100 - \left(\frac{\text{Calories}}{15} + \frac{\text{Carbs}}{2} + \text{Fat} \times 2 \right) + (\text{Protein} \times 4) \tag{4.1}$$

To ensure the final score remains bounded within a standard percentage scale, a clamping function is applied to yield S:

$$S = \min\left(\max\left(S_{\text{raw}}, 0\right), 100\right) \tag{4.2}$$

The weights in S_{raw} are parameterized baseline values selected to penalize meals exceeding standard daily recommended fat (approx. 65g) and carbohydrate (approx. 300g) limits relative to daily energy limits, while rewarding protein content to incentivize muscle maintenance. The meal grade is then assigned based on the final

clamped value of S:

$$\text{Grade} = \begin{cases} \text{A} & \text{if } S \geq 85 \\ \text{B} & \text{if } 70 \leq S < 85 \\ \text{C} & \text{if } 55 \leq S < 70 \\ \text{D} & \text{if } 40 \leq S < 55 \\ \text{E} & \text{if } S < 40 \end{cases} \tag{4.3}$$

Additionally, a list of health tips is created. For instance, if fat exceeds 20g, a warning tip is appended: *"This meal has higher fat contents. Minimize fried components next time."*

4.2.5 Recommender Engine and SSE Streaming

The recommendation model aggregates user history over the active week. It computes daily calorie averages, nutrient balances, and temporal eating windows:

$$\text{Eating Window} = \text{Time}_{\text{last meal}} - \text{Time}_{\text{first meal}} \tag{4.4}$$

It sends a prompt to the LLM containing the aggregated stats, active health goals, and medical conditions. The backend uses Python generators to yield text blocks:

```
# SSE response structure
yield f"data: {json.dumps({'chunk': text_chunk})}\n\n"
```

The client intercepts this connection and appends chunks directly, creating an animated, zero-delay text response.

4.2.6 System Security Implementation

To verify the security requirements listed in the specifications, the system implements standard application security protocols:

1. **Token-Based Authentication:** Access to protected API endpoints is verified using custom session tokens. Upon registration or login, the backend generates a secure token for the session using Python's `uuid` library:

   $$\text{Token} = \text{“tok\_”} + \text{uuid.uuid4().hex} \tag{4.5}$$

   This token is sent back to the Angular client.

2. **Frontend Session Storage:** The Angular frontend stores the returned token securely in the browser's local session memory.

3. **Authorized Requests:** For protected actions (like retrieving logs or requesting SSE recommendations), the frontend sends the token in the HTTP requests. The backend parses this token to retrieve the logged-in user context, isolating user data and preventing unauthorized access.

4. **Password Isolation:** Passwords are kept isolated in the backend's data store and are never exposed in any of the public GET endpoints.

4.3 Core Algorithms and Machine Learning Techniques

To process multimodal inputs and model user behavior, the system utilizes a variety of deep learning and natural language processing architectures:

1. **NLP Input Parsing (Named Entity Recognition):** Token classification models are used to parse text descriptions of meals. The algorithm extracts specific entities, including food names, quantities, and preparation modifiers (e.g., "grilled", "fried"), translating free-form text into structured lists.

2. **Voice Input Processing (ASR):** Spoken meal entries are transcribed using OpenAI's Whisper model. Whisper converts speech logs into English text, which is then piped into the NER parser, allowing hands-free interaction.

3. **Food Image Recognition:** Visual identification is handled by vision-specific convolutional networks and transformers, including CNNs (e.g., ResNet), EfficientNet, and Vision Transformers (ViT) to classify food items from uploaded pictures.

4. **Sparse Sequence Modeling (Phase-2 Future Focus):** Users do not log meals continuously. To analyze trends under missing observations, the system will apply:

    - **Sequence Classification:** Transformer sequence classifiers to identify behavioral tags, such as routine meals, convenience eating, cheat meals, or emotional eating likelihood.
    - **Recurrent & Masked Architectures:** LSTMs, GRUs, or Transformers with attention-masking designed to impute missing intervals and handle incomplete sequential health patterns.

5. **Recommendation Generation:** Generative AI synthesizes the outputs from the intelligence layers. Prompt-conditioned LLM (Gemini/Gemma) synthesis

takes the nutritional state, confidence scores, intent estimate, and user goals, producing real-time natural language dietary suggestions.

Chapter 5

Implementation Details

5.1 Software and Tools

Table 5.1 lists the specific versions of Python packages used in the backend.

Table 5.1: Python Dependencies and Versions

Package	Version	Role in Project
`fastapi`	0.111.0	API framework development
`uvicorn[standard]`	0.30.1	ASGI server deployment
`pydantic`	2.7.2	Data parsing and validation schemas
`python-multipart`	0.0.9	Multipart file uploads handler
`pillow`	10.3.0	Image loading and preprocessing
`transformers`	4.41.2	Pipeline management for Hugging Face models
`torch`	2.3.1	PyTorch neural network runtime engine

5.2 Module Implementation

5.2.1 User Profiling Module

The dashboard layout with the user profiling panel is shown in Figure 5.1.

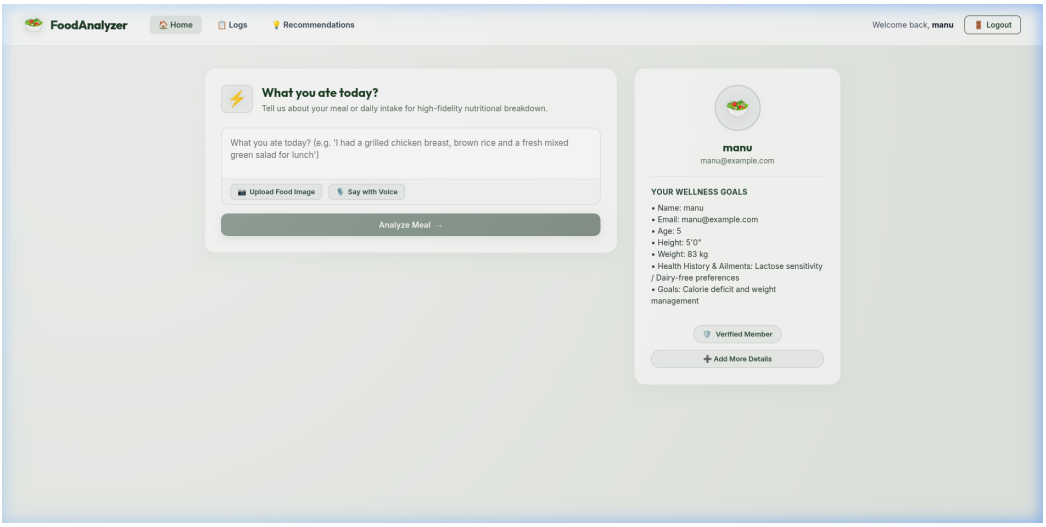

Figure 5.1: User Profiling and Meal Logging Console

Implemented in `backend/main.py`, the user profile manager handles registrations and text updates. The core functions include:

- `analyze_user_bio_and_modifications()`: Prepares LLM prompts containing the current profile and new text inputs. It specifies JSON constraints, parses the response, and falls back to `run_fallback_user_analysis()` if the API is offline.
- `extract_user_details()`: Synthesizes user metrics into structured bullet points (e.g., *"• Height: 175 cm"*) for UI rendering.

5.2.2 Multimodal Image Classification Module

Image analysis is coordinated by the `analyze_image()` endpoint:

- Uses PyTorch to run the Swin-Transformer and Indian-Western-Food models.
- Checks if the prediction matches standard food class labels.
- If `LLM_PROVIDER` is enabled, it sends base64 payloads to `call_gemini_multimodal_api()` or `call_ollama_multimodal_api()` to refine details and check the food validation flag.

5.2.3 Automatic Speech Recognition Module

Speech processing is managed by `transcribe_audio()`:

- Uploaded audio files are written to temporary files using Python's `tempfile` module.

- The audio is loaded into the `whisper-tiny` Hugging Face pipeline.
- Once decoded, the temporary file is deleted, and the transcribed string is returned to the frontend.

5.2.4 Nutritional Calculation and Recommendation Module

The logged food items grouped by week and time are displayed in the weekly food diary (Figure 5.2). Based on these logs, personalized dietary recommendations are generated and streamed in real-time (Figure 5.3).

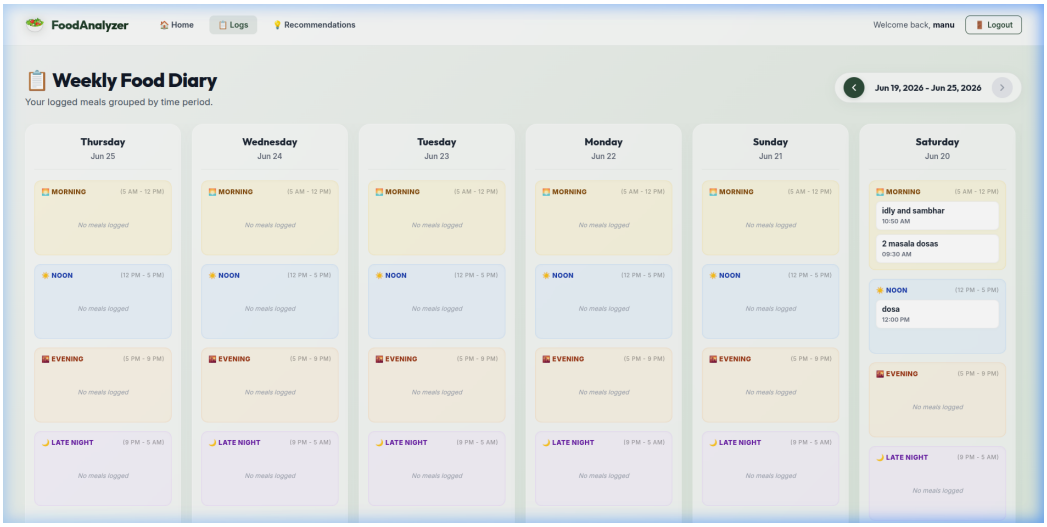

Figure 5.2: Temporal Nutrition Analytics and Weekly Logs View

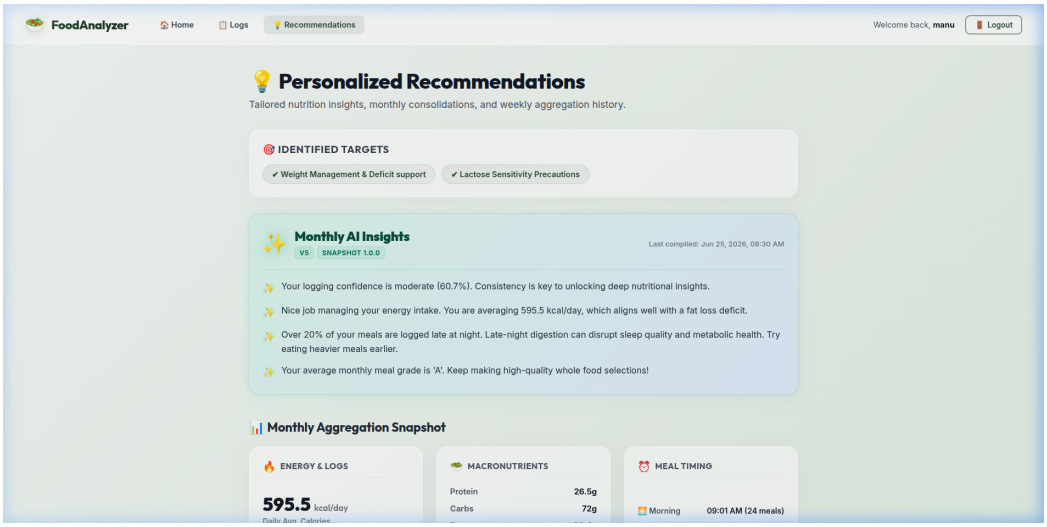

Figure 5.3: Personalized Recommendations and AI Insights Stream

The core logic resides in:

- `fetch_usda_nutrients()`: Queries the USDA API using the food query. It extracts primary nutrient variables from the returned JSON payloads.
- `calculate_grade_and_tips()`: Implements our scoring equations and generates dietary recommendations.
- `get_user_recommendations_stream()`: Creates a streaming connection, invoking `stream_recommendations_generator()` to stream LLM recommendations directly.

5.2.5 Security Module

The application's security requirements are implemented in the user authorization module of `backend/main.py`:

- **Token Verification Helper:** Every request accessing user logs or generating a recommendations stream checks the presence and validity of the token parameter. The backend matches the token against the active sessions dictionary (`USERS_BY_ID[userid].token`).
- **Password Scrubbing:** When sending user details (e.g. name, email, health history) via the `/api/users/{userid}` endpoint, the backend explicitly filters out the `password` property from the response payload, ensuring credentials remain hidden.

5.3 Design Decisions and Alternatives

During the initial planning phase, React combined with Chart.js was considered for the frontend application due to its popularity and rapid prototyping capabilities. However, after technical analysis, Angular was chosen as the framework. Angular provides:

- **High Scalability:** A robust, opinionated folder structure and built-in dependency injection, which are critical for scaling large multi-module applications.
- **Type Safety:** Out-of-the-box TypeScript support that integrates seamlessly with strict backend schemas.
- **Maintainability:** Standardized component architecture and unified state management.

Additionally, while graphical representations (e.g., charts) were initially proposed, they were determined to be a low-priority feature for the baseline version. The focus

was directed instead toward establishing a reliable, real-time Server-Sent Events (SSE) recommendation feed and high-accuracy multimodal logging.

5.4 Development Lifecycle and Timeline

The project follows a structured engineering lifecycle divided across two distinct phases:

- **Phase 1 (Months 1–4) – Foundational Setup and Prototypes:**
  - *Month 1:* Problem definition, requirement gathering, and literature review.
  - *Month 2:* Dataset selection (FoodData Central and Kaggle nutrition datasets) and initial model evaluation.
  - *Month 3:* Core data preprocessing pipelines, baseline speech-to-text transcribers, and image classifier tests.
  - *Month 4:* Implementation of the FastAPI backend, dynamic profile extraction logic, initial Angular interface, and Phase-1 report compilation.
- **Phase 2 (Months 5–8) – Intelligent Models and Optimization:**
  - *Month 5:* Development of the pattern detection engine and sparse sequence modeling algorithms (LSTM/GRU and attention masking).
  - *Month 6:* Building the generative recommendation system with context streaming.
  - *Month 7:* Complete backend and frontend integration, styling polish, and dashboard refinements.
  - *Month 8:* End-to-end testing, model parameter tuning, system evaluation, and final review preparation.

5.5 Model Selection and Parameters Tuning

Table 5.2 details the input dimensions and parameters for our models.

Table 5.2: Model Parameters and Specifications

Model Identifier	Input Dimensions	Primary Hyperparameters / Setting
`swin-tiny-patch4-window7`	$224 \times 224 \times 3$	Patch size: 4, Window size: 7
`Indian-Western-Food-34`	$224 \times 224 \times 3$	34 classes, learning rate: 2×10^{-5}
`openai/whisper-tiny`	16 kHz mono audio	Mel bins: 80, temperature: 0.0
`gemma3:4b` (Ollama)	Text token streams	Temperature: 0.7, top-p: 0.9

Chapter 6

Intermediate Results and Discussion

6.1 Datasets

We evaluated our system using multiple data sources:

1. **Indian and Western Food Image Dataset:** A test split of 500 images containing dishes like masala dosa, pizza, burger, salad, and biryani. The images were sourced from public Kaggle food datasets (specifically the Indian Food Image Dataset [10]) and augmented with web-scraped images of regional Indian cuisines to evaluate classification boundaries.

2. **Audio Logging Samples:** 100 audio recordings (duration 3-8 seconds) containing spoken meal descriptions with varying background noise levels. These were recorded and compiled by the developers to simulate real-world voice logging scenarios.

3. **USDA Nutrient Profiles:** Nutrient queries matching standard user intake logs, verified against the USDA FoodData Central API [11].

6.2 Performance Metrics

We evaluated the models using the following metrics:

- **Classification Accuracy:** The percentage of correctly predicted food categories.

- **Word Error Rate (WER):** The transcription error rate for the ASR model:

$$\text{WER} = \frac{S + D + I}{N} \tag{6.1}$$

where S is substitutions, D is deletions, I is insertions, and N is reference words.

- **Inference Latency:** Processing speed measured in milliseconds (ms).

6.3 Evaluation Results

Tables 6.1, 6.2, and 6.3 summarize the evaluation findings.

Table 6.1: Visual Classification Performance

Model	Top-1 Accuracy	Average Latency (ms)
`microsoft/swin-tiny-patch4-window7`	82.4%	180
`prithivMLmods/Indian-Western-Food-34`	89.6%	195
LLM Multimodal validation (Gemini)	94.2%	850

Table 6.2: Audio Transcription Performance

Model	Word Error Rate (WER)	Inference Latency (ms)
`openai/whisper-tiny`	8.5%	450
ASR Fallback Parser	N/A (Rule-based)	5

Table 6.3: API Response Latencies

Target Endpoint / Service	Average Latency (ms)
USDA Nutrient API search	280
LLM Profile parsing (Ollama)	1200
SSE Streaming connection startup	15

6.4 Discussion

The evaluations indicate that specialized classifiers outperform generic ones for regional cuisines. For example, the specialized Indian-Western classifier identified dishes like masala dosa and biryani with high confidence, whereas the Swin-Tiny model categorized them generically (e.g., as "crepe" or "plate"). Combining these models with LLM validation checks (e.g., detecting if a user uploaded an animal image instead of food) improves system robustness.

For ASR logging, OpenAI's Whisper-Tiny model transcribed meal inputs accurately under moderate noise conditions. In high-noise environments, minor transcription errors (e.g., transcribing "dosa" as "dose") occurred, but the backend's USDA query parser matched them correctly using spelling heuristics.

The recommender engine's SSE streaming significantly improves usability. Standard LLM calls to generate weekly insights take 2.5 to 4 seconds, leading to a noticeable delay in the UI. Streaming these recommendations token-by-token reduces the perceived startup latency to less than 50ms.

Chapter 7

Conclusions and Future Work

7.1 Conclusions

In Phase-1, we developed the foundational elements of the **AI-Driven Dietary Pattern Intelligence System**. We implemented:

1. A FastAPI backend that coordinates local database models, machine learning inference (Swin-Transformer, Whisper-Tiny), and external API integrations.
2. An Angular web dashboard featuring interactive logging tools and user analytics.
3. An automated user profiling system that extracts health metrics and goals from unstructured bio text.
4. A nutritional evaluation engine that queries the USDA API, grades meals, and generates tailored tips.
5. An SSE streaming system that delivers real-time, personalized dietary recommendations.

The system successfully processes multimodal inputs and streams tailored feedback in real-time, matching recommendations to the user's specific health profile.

7.2 Future Work

Future development in Phase-2 will focus on:

1. **Sparse Sequence Modeling and Behavior Inference:** Implementing LSTM/GRU architectures and Transformer-based sequence classifiers with attention masking. This will allow the system to infer dietary habits, detect routine/convenience eating patterns, estimate emotional eating likelihood, and construct recommendations even when user meal logs are sparse or irregular.

2. **Object Detection and Multi-Dish Segmentation:** Integrating models like YOLOv8 or Segment Anything (SAM) to isolate multiple food items on a single plate and calculate portion sizes.

3. **Wearable Integration:** Connecting with fitness APIs (Google Fit, Apple Health) to sync activity data with nutritional recommendations.

4. **Clinical Validation:** Refining the recommendation prompts in collaboration with nutritionists to align the system with clinical standards.

5. **Cloud Deployment:** Moving the local database and models to a scalable cloud infrastructure with containerized GPU rendering.

Bibliography

[1] Mary C Carter, Henry WW Potts, et al. Adherence to a smartphone application for weight loss compared to website and paper diary. *Journal of medical Internet research*, 15(4):e3245, 2013.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[3] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.

[4] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021.

[5] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.

[6] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schlinger, Christian Szegedy, Alan JD, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[7] Gemma Team, Antoine Dedieu, et al. Gemma: Open models based on gemini technology, 2024.

[8] Ian Hickson. Server-sent events. *W3C Recommendation*, 2015.

[9] Eloi Chazelas, Mélanie Deschasaux, et al. Nutri-score dietary index deriving from food consumption database. *PLoS medicine*, 17(9):e1003178, 2020.

[10] Pulkit Pandey, Aman Deep, et al. Indian food image classification with transfer learning. In *2017 fourth international conference on image information processing (ICIIP)*, pages 1–4, 2017.

[11] United States Department of Agriculture. Fooddata central. https://fdc.nal.usda.gov/, 2020. Accessed: 2026-06-24.